Guía básica para trabajar con diccionarios en Python

1. ¿Qué es un diccionario?

Un **diccionario** en Python es una estructura que permite guardar datos usando una relación de **clave** y **valor**.

Una forma sencilla de entenderlo es pensar en una ficha de una persona:

Clave	Valor
nombre	Ana
edad	20
carrera	Informática

En Python se vería así:

```
persona = {
   "nombre": "Ana",
   "edad": 20,
   "carrera": "Informática"
}
```

La clave es el nombre del dato, por ejemplo `"nombre"`.

El valor es la información asociada a esa clave, por ejemplo `"Ana"`.

---

2. ¿Para qué sirven los diccionarios?

Los diccionarios sirven para representar datos más organizados.

Por ejemplo, si usamos una lista:

persona = ["Ana", 20, "Informática"]

El problema es que no queda tan claro qué representa cada dato.

En cambio, con un diccionario:

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}
```

Es mucho más fácil entender la información.

---

3. Crear un diccionario

Para crear un diccionario usamos llaves `{}`.

```
estudiante = {
    "nombre": "Carlos",
    "edad": 22,
    "nota": 6.2
}
```

También podemos crear un diccionario vacío:

```
estudiante = {}
```

Esto es útil cuando queremos agregar información más adelante.

---

4. Claves y valores

Cada elemento de un diccionario tiene dos partes:

"nombre": "Carlos"

La parte izquierda es la **clave**:

"nombre"

La parte derecha es el **valor**:

"Carlos"

Un diccionario puede guardar distintos tipos de datos:

```
producto = {
    "nombre": "Notebook",
    "precio": 550000,
    "disponible": True,
    "descuento": 0.15
}
```

---

5. Acceder a un valor del diccionario

Para acceder a un valor usamos su clave.

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}

print(persona["nombre"])
print(persona["edad"])
```

Salida:

```
Ana
20
```

En este caso:

```
persona["nombre"]
```

significa: obtener el valor asociado a la clave `"nombre"`.

---

6. Error común al acceder a una clave que no existe

Si intentamos acceder a una clave inexistente, Python genera un error.

```
persona = {
    "nombre": "Ana",
    "edad": 20
}

print(persona["correo"])
```

Esto genera error porque la clave `"correo"` no existe.

Para evitar ese problema, podemos usar `get()`.

```
persona = {
    "nombre": "Ana",
    "edad": 20
}

print(persona.get("correo"))
```

Salida:

None

También podemos indicar un valor por defecto:

```
persona = {
    "nombre": "Ana",
    "edad": 20
}

print(persona.get("correo", "No registrado"))
```

Salida:

No registrado

---

7. Modificar valores de un diccionario

Los diccionarios son modificables.

```
persona = {
```

```
    "nombre": "Ana",
    "edad": 20
}

persona["edad"] = 21

print(persona)
```

Salida:

```
{'nombre': 'Ana', 'edad': 21}
```

En este caso, el valor de la clave `"edad"` cambió de `20` a `21`.

---

8. Agregar nuevos datos

Para agregar un nuevo dato, usamos una nueva clave.

```
persona = {
    "nombre": "Ana",
    "edad": 20
}

persona["correo"] = "ana@email.com"

print(persona)
```

Salida:

```
{'nombre': 'Ana', 'edad': 20, 'correo': 'ana@email.com'}
```

Si la clave no existe, Python la crea.

Si la clave ya existe, Python actualiza su valor.

---

9. Eliminar datos de un diccionario

Eliminar con `pop()`

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "correo": "ana@email.com"
}

correo_eliminado = persona.pop("correo")

print(persona)
print(correo_eliminado)
```

Salida:

```
{'nombre': 'Ana', 'edad': 20}
ana@email.com
```

`pop()` elimina la clave indicada y devuelve su valor.

Eliminar con `del`

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "correo": "ana@email.com"
}

del persona["correo"]

print(persona)
```

Salida:

```
{'nombre': 'Ana', 'edad': 20}
```

Vaciar un diccionario con `clear()`

```
persona = {
    "nombre": "Ana",
    "edad": 20
```

```
}

persona.clear()

print(persona)
```

Salida:

```
{}
```

---

10. Verificar si una clave existe

Podemos usar `in` para saber si una clave existe en un diccionario.

```
persona = {
    "nombre": "Ana",
    "edad": 20
}

if "nombre" in persona:
    print("La clave nombre existe")
else:
    print("La clave nombre no existe")
```

Salida:

```
La clave nombre existe
```

También podemos usar `not in`.

```
if "correo" not in persona:
    print("La persona no tiene correo registrado")
```

---

11. Recorrer un diccionario

Una operación muy común es recorrer un diccionario usando un ciclo `for`.

Recorrer claves

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}

for clave in persona:
    print(clave)
```

Salida:

```
nombre
edad
carrera
```

Por defecto, Python recorre las claves del diccionario.

Recorrer valores

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}

for valor in persona.values():
    print(valor)
```

Salida:

```
Ana
20
Informática
```

Recorrer claves y valores

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}
```

```
for clave, valor in persona.items():
    print(clave, ":", valor)
```

Salida:

```
nombre : Ana
edad : 20
carrera : Informática
```

Este es uno de los recorridos más usados con diccionarios.

---

12. Obtener solo las claves

Podemos obtener todas las claves usando `keys()`.

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}

print(persona.keys())
```

Salida:

```
dict_keys(['nombre', 'edad', 'carrera'])
```

También podemos recorrerlas:

```
for clave in persona.keys():
    print(clave)
```

---

13. Obtener solo los valores

Podemos obtener todos los valores usando `values()`.

```
persona = {
    "nombre": "Ana",
```

```
    "edad": 20,
    "carrera": "Informática"
}

print(persona.values())
```

Salida:

```
dict_values(['Ana', 20, 'Informática'])
```

También podemos recorrerlos:

```
for valor in persona.values():
    print(valor)
```

---

14. Obtener claves y valores juntos

Podemos obtener las claves y valores usando `items()`.

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}

print(persona.items())
```

Salida:

```
dict_items([('nombre', 'Ana'), ('edad', 20), ('carrera', 'Informática')])
```

Lo más común es usarlo con `for`:

```
for clave, valor in persona.items():
    print(clave, valor)
```

---

15. Cantidad de elementos en un diccionario

Para saber cuántos pares clave-valor tiene un diccionario, usamos `len()`.

```
persona = {
    "nombre": "Ana",
    "edad": 20,
    "carrera": "Informática"
}

print(len(persona))
```

Salida:

```
3
```

---

16. Diccionario con datos ingresados por el usuario

Podemos pedir datos al usuario y guardarlos en un diccionario.

```
estudiante = {}

estudiante["nombre"] = input("Ingrese el nombre del estudiante: ")
estudiante["edad"] = int(input("Ingrese la edad del estudiante: "))
estudiante["carrera"] = input("Ingrese la carrera del estudiante: ")

print(estudiante)
```

Ejemplo de ejecución:

```
Ingrese el nombre del estudiante: Valentina
Ingrese la edad del estudiante: 21
Ingrese la carrera del estudiante: Informática
{'nombre': 'Valentina', 'edad': 21, 'carrera': 'Informática'}
```

---

17. Ejemplo práctico: ficha de producto

```
producto = {
    "nombre": "Mouse",
    "precio": 12000,
    "stock": 15
}

print("Producto:", producto["nombre"])
print("Precio:", producto["precio"])
print("Stock:", producto["stock"])
```

Salida:

```
Producto: Mouse
Precio: 12000
Stock: 15
```

También podemos actualizar el stock:

```
producto["stock"] = producto["stock"] - 1

print(producto)
```

Salida:

```
{'nombre': 'Mouse', 'precio': 12000, 'stock': 14}
```

---

18. Ejemplo práctico: calcular total de compra

```
producto = {
    "nombre": "Teclado",
    "precio": 25000,
    "cantidad": 3
}

total = producto["precio"] * producto["cantidad"]

print("Producto:", producto["nombre"])
print("Total a pagar:", total)
```

Salida:

Producto: Teclado
Total a pagar: 75000

---

19. Diccionarios dentro de una lista

Es muy común trabajar con una lista que contiene varios diccionarios.

Por ejemplo, una lista de estudiantes:

```
estudiantes = [
    {
        "nombre": "Ana",
        "edad": 20,
        "nota": 6.5
    },
    {
        "nombre": "Luis",
        "edad": 22,
        "nota": 5.8
    },
    {
        "nombre": "Camila",
        "edad": 19,
        "nota": 6.9
    }
]
```

Cada estudiante es un diccionario.

La lista completa contiene varios estudiantes.

---

20. Recorrer una lista de diccionarios

```
estudiantes = [
    {"nombre": "Ana", "edad": 20, "nota": 6.5},
    {"nombre": "Luis", "edad": 22, "nota": 5.8},
    {"nombre": "Camila", "edad": 19, "nota": 6.9}
```

```
]

for estudiante in estudiantes:
    print("Nombre:", estudiante["nombre"])
    print("Edad:", estudiante["edad"])
    print("Nota:", estudiante["nota"])
    print("-------------")
```

Salida:

```
Nombre: Ana
Edad: 20
Nota: 6.5
-------------
Nombre: Luis
Edad: 22
Nota: 5.8
-------------
Nombre: Camila
Edad: 19
Nota: 6.9
-------------
```

Este formato es muy usado en sistemas reales, por ejemplo:

- Listado de usuarios.
- Catálogo de productos.
- Registro de ventas.
- Datos recibidos desde una API.
- Información obtenida desde una base de datos.

---

21. Buscar un dato en una lista de diccionarios

Supongamos que queremos buscar un estudiante por nombre.

```
estudiantes = [
    {"nombre": "Ana", "edad": 20, "nota": 6.5},
    {"nombre": "Luis", "edad": 22, "nota": 5.8},
    {"nombre": "Camila", "edad": 19, "nota": 6.9}
]
```

```
buscado = input("Ingrese el nombre a buscar: ")

for estudiante in estudiantes:
    if estudiante["nombre"] == buscado:
        print("Estudiante encontrado:")
        print(estudiante)
```

Ejemplo de ejecución:

```
Ingrese el nombre a buscar: Luis
Estudiante encontrado:
{'nombre': 'Luis', 'edad': 22, 'nota': 5.8}
```

---

22. Mejorar la búsqueda con una variable de control

El ejemplo anterior funciona, pero si no encuentra al estudiante, no muestra nada.

Podemos mejorarlo así:

```
estudiantes = [
    {"nombre": "Ana", "edad": 20, "nota": 6.5},
    {"nombre": "Luis", "edad": 22, "nota": 5.8},
    {"nombre": "Camila", "edad": 19, "nota": 6.9}
]

buscado = input("Ingrese el nombre a buscar: ")

encontrado = False

for estudiante in estudiantes:
    if estudiante["nombre"] == buscado:
        print("Estudiante encontrado:")
        print(estudiante)
        encontrado = True

if encontrado == False:
    print("Estudiante no encontrado")
```

Ejemplo de ejecución:

```
Ingrese el nombre a buscar: Pedro
```

Estudiante no encontrado

---

23. Agregar diccionarios a una lista

Podemos crear varios diccionarios y agregarlos a una lista.

```
estudiantes = []

for i in range(3):
    estudiante = {}

    estudiante["nombre"] = input("Ingrese nombre: ")
    estudiante["edad"] = int(input("Ingrese edad: "))
    estudiante["nota"] = float(input("Ingrese nota: "))

    estudiantes.append(estudiante)

print(estudiantes)
```

Ejemplo de ejecución:

```
Ingrese nombre: Ana
Ingrese edad: 20
Ingrese nota: 6.5
Ingrese nombre: Luis
Ingrese edad: 22
Ingrese nota: 5.8
Ingrese nombre: Camila
Ingrese edad: 19
Ingrese nota: 6.9

[{'nombre': 'Ana', 'edad': 20, 'nota': 6.5}, {'nombre': 'Luis', 'edad': 22, 'nota': 5.8}, {'nombre': 'Camila', 'edad': 19, 'nota': 6.9}]
```

---

24. Mostrar datos de forma más ordenada

```
estudiantes = [
    {"nombre": "Ana", "edad": 20, "nota": 6.5},
```

```
    {"nombre": "Luis", "edad": 22, "nota": 5.8},
    {"nombre": "Camila", "edad": 19, "nota": 6.9}
]

for estudiante in estudiantes:
    print("Nombre:", estudiante["nombre"])
    print("Edad:", estudiante["edad"])
    print("Nota:", estudiante["nota"])
    print()
```

Salida:

```
Nombre: Ana
Edad: 20
Nota: 6.5

Nombre: Luis
Edad: 22
Nota: 5.8

Nombre: Camila
Edad: 19
Nota: 6.9
```

---

25. Ejemplo práctico: sistema simple de productos

```
productos = []

while True:
    print("1. Agregar producto")
    print("2. Mostrar productos")
    print("3. Salir")

    opcion = input("Seleccione una opción: ")

    if opcion == "1":
        producto = {}

        producto["nombre"] = input("Ingrese nombre del producto: ")
        producto["precio"] = int(input("Ingrese precio del producto: "))
        producto["stock"] = int(input("Ingrese stock del producto: "))
```

```
        productos.append(producto)

    elif opcion == "2":
        for producto in productos:
            print("Nombre:", producto["nombre"])
            print("Precio:", producto["precio"])
            print("Stock:", producto["stock"])
            print("-------------")

    elif opcion == "3":
        print("Programa finalizado")
        break

    else:
        print("Opción no válida")
```

Este programa permite:

- Agregar productos.
- Guardar cada producto como diccionario.
- Guardar todos los productos dentro de una lista.
- Mostrar los productos registrados.

---

26. Errores comunes al trabajar con diccionarios

Error 1: Usar una clave que no existe

```
persona = {
    "nombre": "Ana"
}

print(persona["edad"])
```

Esto genera error porque `"edad"` no existe.

Forma segura:

```
print(persona.get("edad", "Edad no registrada"))
```

Error 2: Confundir listas con diccionarios

Una lista usa índices numéricos:

```
frutas = ["manzana", "pera"]

print(frutas[0])
```

Un diccionario usa claves:

```
persona = {
    "nombre": "Ana"
}

print(persona["nombre"])
```

No sería correcto hacer esto:

```
print(persona[0])
```

porque el diccionario no tiene una clave llamada `0`.

Error 3: Olvidar las comas

Incorrecto:

```
persona = {
    "nombre": "Ana"
    "edad": 20
}
```

Correcto:

```
persona = {
    "nombre": "Ana",
    "edad": 20
}
```

Cada par clave-valor debe separarse con coma.

Error 4: Repetir claves

```
persona = {
    "nombre": "Ana",
    "nombre": "Camila"
}

print(persona)
```

Salida:

```
{'nombre': 'Camila'}
```

Si repetimos una clave, Python se queda con el último valor asignado.

---

27. Diferencia entre lista y diccionario

Lista	Diccionario
Usa índices numéricos	Usa claves
Se accede con `lista[0]`	Se accede con `diccionario["clave"]`
Sirve para datos ordenados o secuenciales	Sirve para datos con significado
Ejemplo: lista de notas	Ejemplo: ficha de estudiante

Ejemplo de lista:

```
notas = [5.5, 6.0, 4.8]
```

Ejemplo de diccionario:

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}
```

---

28. ¿Cuándo usar diccionarios?

Conviene usar diccionarios cuando los datos tienen una estructura clara.

Por ejemplo:

```
usuario = {
    "id": 1,
    "nombre": "Pedro",
    "correo": "pedro@email.com",
    "activo": True
}
```

Otros ejemplos:

```
auto = {
    "marca": "Toyota",
    "modelo": "Yaris",
    "anio": 2022
}

libro = {
    "titulo": "Python básico",
    "autor": "María López",
    "paginas": 180
}

venta = {
    "producto": "Mouse",
    "cantidad": 2,
    "precio_unitario": 12000
}
```

---

29. Ejercicios propuestos

Ejercicio 1: Ficha personal

Crea un diccionario llamado `persona` con las siguientes claves:

- `nombre`
- `edad`
- `ciudad`

Luego muestra cada dato por pantalla.

Ejercicio 2: Producto

Crea un diccionario llamado `producto` con:

- `nombre`
- `precio`
- `stock`

Luego calcula el valor total del inventario multiplicando `precio * stock`.

Ejercicio 3: Modificar datos

Crea un diccionario de un estudiante con:

- `nombre`
- `nota`

Luego modifica la nota y muestra el diccionario actualizado.

Ejercicio 4: Agregar nueva clave

Crea un diccionario con nombre y edad.

Después agrega una nueva clave llamada `correo`.

Ejercicio 5: Eliminar una clave

Crea un diccionario de un producto.

Luego elimina la clave `stock`.

Ejercicio 6: Verificar existencia

Crea un diccionario de usuario.

Pide al usuario ingresar una clave.

El programa debe indicar si esa clave existe o no en el diccionario.

Ejercicio 7: Recorrer diccionario

Crea un diccionario con datos de una película:

- `titulo`
- `director`
- `anio`
- `genero`

Luego recorre el diccionario mostrando claves y valores.

Ejercicio 8: Lista de estudiantes

Crea una lista con 3 diccionarios.

Cada diccionario debe representar un estudiante con:

- `nombre`
- `edad`
- `nota`

Luego muestra todos los estudiantes usando un ciclo `for`.

Ejercicio 9: Buscar producto

Crea una lista de productos.

Cada producto debe tener:

- `nombre`
- `precio`
- `stock`

Pide al usuario ingresar el nombre de un producto y muestra sus datos si existe.

Ejercicio 10: Sistema de registro

Crea un programa que permita ingresar varios estudiantes.

Cada estudiante debe guardar:

- `nombre`
- `edad`
- `nota`

El programa debe permitir ingresar estudiantes hasta que el usuario escriba `"salir"`.

Al final, muestra todos los estudiantes registrados.

---

30. Resumen de funciones y métodos importantes

Función o método	¿Para qué sirve?	Ejemplo
`diccionario["clave"]`	Accede a un valor	`persona["nombre"]`
`get()`	Accede de forma segura	`persona.get("edad")`
`keys()`	Obtiene las claves	`persona.keys()`
`values()`	Obtiene los valores	`persona.values()`
`items()`	Obtiene claves y valores	`persona.items()`
`pop()`	Elimina una clave	`persona.pop("edad")`
`clear()`	Vacía el diccionario	`persona.clear()`
`len()`	Cuenta pares clave-valor	`len(persona)`
`in`	Verifica si una clave existe	`"nombre" in persona`

Cierre

Los diccionarios son una herramienta fundamental en Python porque permiten representar información de manera clara y organizada.

Mientras las listas son útiles para guardar varios elementos, los diccionarios permiten guardar datos con significado.

Por ejemplo, una lista puede guardar varias notas:

notas = [5.5, 6.0, 6.8]

Pero un diccionario puede representar una entidad completa:

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "nota": 6.5
}
```

Dominar los diccionarios es clave para avanzar hacia temas como funciones, archivos, bases de datos, APIs, JSON y programación orientada a objetos.